How-To: LLM Security Analyzer

ragent normally decides whether a tool call may run from its **static permission rules** alone: the allow/deny/ask ruleset, the file and directory guards, and the 7-layer bash validator. The LLM security analyzer adds an optional second opinion: a model is shown each proposed tool action and returns an `allow` / `ask` / `deny` verdict with a rationale, and that verdict is applied as a **tightening-only** layer over the static rules.

The analyzer never widens access. It can refuse a call the static rules would have allowed, it can satisfy a rule-less prompt, but it can never override an explicit policy `deny`, and when it fails it degrades to the ordinary interactive prompt.

> **Scope.** This guide covers the analyzer mode only. For the static permission pipeline and the 7-layer bash model see `docs/howtos/permissions.md`. For lifecycle hooks that can also veto a tool call see `docs/howtos/hooks.md`. For the raw config fields see `docs/howtos/config.md` section 7.39.

Table of Contents

- 1. What It Is
- 2. Enabling It
- 3. The Verdict
- 4. Tightening-Only Semantics
- 5. Where It Sits in the Pipeline
- 6. Model and Timeout
- 7. Fail-Safe Behaviour
- 8. Verdict Parsing
- 9. Interaction With Hooks, YOLO, and Autopilot
- 10. Slash Command Reference
- 11. Configuration Reference
- 12. Worked Examples
- 13. See Also

---

1. What It Is

The security analyzer is an **opt-in permission mode** (spec `openhands` FR-006, FR-016, FR-017). When it is enabled, each proposed tool action is evaluated by an LLM that returns:

- a **verdict** - `allow`, `ask`, or `deny`;
- a **rationale** - a one-sentence justification shown to you.

The verdict and rationale are published as an `Event::SecurityVerdict` on the event bus **before** the action is permitted or refused (FR-017), so the reason a call was allowed, denied, or escalated to a prompt is always visible in the TUI.

Unlike the static rules, which are deterministic and cheap, the analyzer makes one extra LLM call per tool action. That is the trade-off: broader contextual judgment in exchange for latency and cost. Because the analyzer is tightening-only, the static rules remain the authority - the analyzer can only make the outcome stricter or escalate to a prompt.

OpenHands parity note. The OpenHands source exposes this as an `--llm-approve` flag and a `SecurityAnalyzer` interface. ragent implements the same capability as a config-gated permission mode with a `/security` TUI surface; there is no `--llm-approve` CLI flag. ragent already ships the static permission rules and the 7-layer bash validation that the analyzer layers on top of.

---

2. Enabling It

The analyzer is off by default. Enable it either from the TUI:

```
/security on
```

or by adding the config block to `ragent.json`:

```
{
  "security_analyzer": {
    "enabled": true,
    "model": { "provider_id": "ollama", "model_id": "qwen2.5:1.5b" },
    "timeout_secs": 20
  }
}
```

`/security on` and `/security off` persist `security_analyzer.enabled` to the loaded config file and invalidate the per-turn config cache, so the change takes effect on the next turn. `/security status` reports the effective state, the model, the timeout, and how many verdicts have been published this session.

---

3. The Verdict

The analyzer returns exactly one of three verdicts:

Verdict	Meaning	Effect
`allow`	The action is clearly safe (read-only, inside the project, no destructive or exfiltration risk).	May satisfy a rule-less prompt; never overrides an explicit policy `deny`.
`ask`	The action is plausibly legitimate but risky, ambiguous, or irreversible.	Falls through to the ordinary interactive prompt.
`deny`	The action is dangerous (destructive, secret access, exfiltration, disabling safety controls).	Hard denial; the action does not run.

The analyzer is instructed to prefer `ask` when uncertain and to never `allow` a destructive, secret-exposing, or security-disabling action.

---

4. Tightening-Only Semantics

The analyzer is a **tightening-only** layer. Its verdict is combined with the static decision as follows (implemented at the permission gate in `crates/ragent-agent/src/session/permissions.rs`):

- **`deny` is a hard denial (FR-006).** The action does not run. The rationale is returned to the model as the tool result - text like `Action denied by security analyzer: <rationale>` - so the model can correct course rather than retry blindly.
- **`allow` can satisfy a bare `ask`.** When no explicit policy rule matched the request (the static result was `ask`), no destructive-action checkpoint is forcing a prompt, and no directory denylist entry matched, the analyzer's `allow` is honoured and the action proceeds without prompting.
- **`allow` never overrides an explicit `deny`.** A configured policy `deny`, a directory denylist hit, or a forced destructive-action checkpoint is decided before the analyzer's approval is considered. The analyzer cannot loosen any of them.
- **`ask`, and any analyzer failure, escalate to the normal flow.** The ordinary interactive prompt runs exactly as it would without the analyzer.

The key invariant: the analyzer can move an outcome *towards* restriction (allow to ask, ask to deny) or resolve a rule-less prompt, but it can never move an outcome away from a restriction the static rules already imposed.

---

5. Where It Sits in the Pipeline

A tool action passes through several gates in order. The analyzer runs **after** the `PreToolUse` hooks and **before** the tool executes:

```
Proposed tool call
   |
   v
Static permission pipeline   -- hardwired rules, auto-approve flags,
   |                            directory lists, PermissionChecker rules
   v
PreToolUse hooks (FR-016)    -- exit 2 blocks, exit 1 warns
   |
   v
LLM security analyzer        -- allow / ask / deny + rationale
   |                            (published as Event::SecurityVerdict)
   v
Permission gate              -- combine verdict with the static decision
   |
   v
Tool execution (or denial / prompt)
```

Running after the hooks means a hook that already blocked the call short-circuits before any analyzer LLM call is made. Running before execution means the verdict is always published before the action is permitted or refused (FR-017).

The analyzer also sits **upstream of the execution backend**. Whether tools run on the host (`local`) or in a container/remote backend, the permission check, the path-containment guard, the always-allowed codeindex hardwiring, and the 7-layer bash validator all run in the session tool-dispatch path before any

backend is handed the call. Selecting a sandbox changes *where* a tool runs, never *whether* it is allowed (FR-002, FR-037).

---

6. Model and Timeout

Field	Type	Default	Description
`enabled`	`bool`	`false`	Whether the analyzer evaluates proposed tool actions.
`model`	`{ provider_id, model_id }`	`None`	Optional model override for the analyzer call. When `None`, the session's primary model is used.
`timeout_secs`	integer	`20`	Wall-clock budget for one analyzer call. A verdict that does not arrive within the budget degrades to `ask`.

Point `model` at a fast/cheap model (for example a small local Ollama model) to keep the per-action latency low. The analyzer request carries no tools, uses `temperature: 0` for determinism, caps output at a few hundred tokens, and sends a single user message naming the tool and its JSON input.

If the configured provider is not registered, or the provider's API key or client cannot be resolved, the analyzer is disabled for that turn with a log warning and the normal permission flow takes over. The analyzer never blocks a turn on its own availability.

---

7. Fail-Safe Behaviour

Any failure to obtain a real verdict degrades to `ask` via `SecurityVerdict::fail_safe`. Covered failure modes:

- the section is absent or `enabled` is `false`;
- the provider, API key, or client cannot be resolved;
- the provider returns an error mid-stream;
- the stream stalls past `timeout_secs`;
- the response is empty or cannot be parsed.

In every case the reason is recorded in the rationale and the caller falls back to the standard interactive flow. A broken analyzer can never silently broaden access - at worst it adds one prompt.

---

8. Verdict Parsing

`parse_verdict` accepts the analyzer's answer in several shapes and always fails safe:

1. **Strict JSON** - a `{"verdict": ..., "rationale": ...}` object anywhere in the response body is extracted (balanced-brace scan that respects string escapes) and used directly. Recognised labels are `allow`/`allowed`, `ask`/`confirm`/`prompt`, and `deny`/`denied`/`block`/`blocked`.

2. **Prose fallback** - when no parsable JSON object is present, the whole response is scanned for a verdict keyword. Order matters: an explicit `deny` wins over any incidental mention of `allow`, then `ask`, then `allow`.
3. **Fail-safe** - an empty or unrecognised response yields `ask` with a rationale explaining that the analyzer was unavailable or unrecognised.

Because the prose scan treats `deny` as the strongest keyword, a model that wraps its answer in chatty prose cannot accidentally turn a refusal into an approval.

---

9. Interaction With Hooks, YOLO, and Autopilot

- `PreToolUse` **hooks (FR-016).** Independent and unchanged. A hook that exits with code 2 blocks the action and returns its stderr as the tool result; exit code 1 warns and continues. Because the analyzer runs after the hooks, a blocking hook means no analyzer call is made.
- **YOLO mode and auto-approve flags.** `--yes` / `auto_approve` and YOLO mode short-circuit the interactive prompt in the static pipeline. The analyst still runs and can still `deny` a call. A forced destructive-action checkpoint is enforced regardless of auto-approval.
- **Destructive-action checkpoints.** A forced checkpoint overrides a static `allow` and is decided before the analyzer's `allow` is consulted, so an analyzer approval can never clear a checkpoint.
- `/security revert`**.** Because a false `deny` is disruptive, `revert` is an alias for `off`: it disables the analyzer immediately (and persists `security_analyzer.enabled: false`) so you can get out of a bad denial without hand-editing config.

---

10. Slash Command Reference

```
/security status      # effective state, model, timeout, verdict count (bare form)
/security on          # enable and persist security_analyzer.enabled: true
/security off         # disable and persist security_analyzer.enabled: false
/security revert      # alias for off - the escape hatch for a false denial
/security help        # usage table
```

`/security` with no argument is equivalent to `/security status`. The alias `security-analyzer` dispatches to the same handler.

---

11. Configuration Reference

```
{
  "security_analyzer": {
    "enabled": true,
    "model": { "provider_id": "ollama", "model_id": "qwen2.5:1.5b" },
    "timeout_secs": 20
  }
}
```

Absent, the section defaults to disabled and the static rules decide alone with no LLM call. See `docs/howtos/config.md` section 7.39 for the full field table.

---

12. Worked Examples

Enable the analyzer pointed at a cheap local model:

```
{
  "security_analyzer": {
    "enabled": true,
    "model": { "provider_id": "ollama", "model_id": "qwen2.5:1.5b" }
  }
}
```

Toggle it from the TUI and check state:

```
/security on
/security status
```

Recover from a false denial:

```
/security revert
```

Run with the session's own model and a shorter budget:

```
{
  "security_analyzer": {
    "enabled": true,
    "timeout_secs": 8
  }
}
```

---

13. See Also

- `docs/howtos/permissions.md` - the static permission pipeline, the 7-layer bash model, autopilot, and YOLO mode.
- `docs/howtos/hooks.md` - `PreToolUse` / `PostToolUse` hooks and their precedence.
- `docs/howtos/config.md` section 7.39 - the `security_analyzer` config fields.
- `specs/openhands/SPEC.md` FR-006, FR-016, FR-017 - the requirements.